

## Contents

|          |                                  |          |
|----------|----------------------------------|----------|
| <b>1</b> | <b>ManyToOne</b>                 | <b>1</b> |
| 1.1      | Pedido . . . . .                 | 1        |
| 1.2      | Fecha . . . . .                  | 1        |
| 1.2.1    | DATETIME y TIMESTAMP . . . . .   | 1        |
| 1.2.2    | TIMEZONE . . . . .               | 2        |
| 1.2.3    | Helpers . . . . .                | 4        |
| 1.3      | Problemas de recursión . . . . . | 4        |
| 1.3.1    | SET y LIST . . . . .             | 6        |

## 1 ManyToOne

### 1.1 Pedido

### 1.2 Fecha

La **primera estrategia**, donde se establece la fecha y hora en el momento de la creación del objeto directamente en la entidad, es más sencilla y directa. Es útil cuando sólo necesitas este comportamiento para una entidad específica y no necesitas más funcionalidades de auditoría.

La **segunda estrategia**, utilizando las anotaciones de auditoría de Spring Data JPA, es más flexible y poderosa. Te permite controlar automáticamente las fechas de creación y modificación a nivel de aplicación, y también te proporciona otras funcionalidades de auditoría, como el seguimiento del usuario que creó o modificó la entidad. Sin embargo, es un poco más compleja y requiere una configuración adicional.

Por lo tanto, si sólo se necesit establecer la fecha y hora de creación y no necesitas otras funcionalidades de auditoría, la primera estrategia puede ser suficiente. Pero si necesitas más funcionalidades de auditoría o quieres aplicar este comportamiento a varias entidades de manera consistente, entonces la segunda estrategia sería más adecuada.

#### 1.2.1 DATETIME y TIMESTAMP

La elección entre **DATETIME** y **TIMESTAMP** depende en gran medida del sistema de gestión de bases de datos (DBMS) que estés utilizando y de tus necesidades específicas.

En **PostgreSQL**, no hay un tipo de datos **DATETIME**. En su lugar, puedes usar **TIMESTAMP** o **TIMESTAMPTZ**. **TIMESTAMP** es sin zona horaria y **TIMESTAMPTZ**

es con zona horaria. Si necesitas tener en cuenta las zonas horarias, deberías usar `TIMESTAMP`.

En **MySQL**, tanto `DATETIME` como `TIMESTAMP` son tipos de datos válidos, pero se comportan de manera diferente. `DATETIME` almacena fechas y horas sin tener en cuenta las zonas horarias, mientras que `TIMESTAMP` convierte las fechas y horas a UTC para su almacenamiento, y las convierte de nuevo a la zona horaria local para su recuperación.

En tu caso, como estás utilizando `LocalDateTime` en tu entidad, que no tiene información de zona horaria, puedes usar `TIMESTAMP` en PostgreSQL y `DATETIME` en MySQL.

Por lo tanto, tu anotación de columna para la fecha y hora en la entidad `Pedido` se vería así para PostgreSQL:

```
@Column(name = "fecha_hora", columnDefinition = "TIMESTAMP")
private LocalDateTime fechaHora = LocalDateTime.now();
```

Y así para MySQL:

```
@Column(name = "fecha_hora", columnDefinition = "DATETIME")
private LocalDateTime fechaHora = LocalDateTime.now();
```

Espero que esto te ayude. Si tienes alguna otra pregunta, no dudes en preguntar.

### 1.2.2 TIMEZONE

Si quieres tener en cuenta la zona horaria, deberías usar `ZonedDateTime` en lugar de `LocalDateTime`. `ZonedDateTime` es una clase de fecha y hora con una zona horaria en el formato ISO-8601, como '2007-12-03T10:15:30+01:00 Europe/Paris'.

Aquí te muestro cómo puedes cambiar tu entidad `Pedido` para usar `ZonedDateTime`:

```
package com.example.producto.modelo;

import jakarta.persistence.*;
import lombok.Data;
import org.springframework.data.annotation.CreatedDate;
import org.springframework.data.jpa.domain.support.AuditingEntityListener;

import java.time.ZonedDateTime;
```

```

import java.util.List;

@Entity
@Data
@Table(name = "pedido")
@EntityListeners(AuditingEntityListener.class)
public class Pedido {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String cliente;

    @OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL)
    private List<LineaPedido> lineasPedido;

    @CreatedDate
    @Column(name = "fecha_hora", columnDefinition = "TIMESTAMP WITH TIME ZONE")
    private ZonedDateTime fechaHora = ZonedDateTime.now();

    public double getTotal() {
        return lineasPedido.stream().mapToDouble(LineaPedido::getSubtotal).sum();
    }

    public void addLineaPedido(LineaPedido lp) {
        lineasPedido.add(lp);
        lp.setPedido(this);
    }

    public void removeLineaPedido(LineaPedido lp) {
        lineasPedido.remove(lp);
        lp.setPedido(null);
    }
}

```

En este código, he cambiado `LocalDateTime` a `ZonedDateTime` y he actualizado la definición de la columna a `TIMESTAMP WITH TIME ZONE` para reflejar esto. Ahora, `fechaHora` se establecerá automáticamente en la fecha y hora actuales, incluyendo la zona horaria.

Espero que esto te ayude. Si tienes alguna otra pregunta, no dudes en preguntar.

### 1.2.3 Helpers

Estos métodos son útiles para mantener la coherencia en tu base de datos y facilitar la manipulación de las relaciones entre entidades. Los utilizarías cuando necesites añadir o eliminar `LineaPedido` de un `Pedido`.

Estos métodos son conocidos como métodos "helper" o de ayuda y se utilizan para manejar la relación bidireccional entre `Pedido` y `LineaPedido`.

El método `addLineaPedido(LineaPedido lp)` se utiliza para añadir una `LineaPedido` a un `Pedido`. Cuando añades una `LineaPedido` a un `Pedido`, también quieres asegurarte de que la `LineaPedido` sabe a qué `Pedido` pertenece. Por eso, dentro del método, después de añadir la `LineaPedido` a la lista de `lineas`, también estableces el `Pedido` de la `LineaPedido`.

De manera similar, el método `removeLineaPedido(LineaPedido lp)` se utiliza para eliminar una `LineaPedido` de un `Pedido`. Cuando eliminas una `LineaPedido` de un `Pedido`, también quieres asegurarte de que la `LineaPedido` ya no hace referencia a ese `Pedido`. Por eso, dentro del método, después de eliminar la `LineaPedido` de la lista de `lineas`, también estableces el `Pedido` de la `LineaPedido` a `null`.

Estos métodos son útiles para mantener la coherencia en tu base de datos y facilitar la manipulación de las relaciones entre entidades. Los utilizarías cuando necesites añadir o eliminar `LineaPedido` de un `Pedido`.

## 1.3 Problemas de recursión

Sí, puedes experimentar un problema de recursión infinita al serializar entidades con relaciones bidireccionales a JSON. Esto se debe a que cada entidad tiene una referencia a la otra, lo que puede llevar a un bucle infinito durante la serialización.

Para resolver este problema, puedes usar la anotación `@JsonManagedReference` y `@JsonBackReference` de la biblioteca Jackson, que es la biblioteca de serialización/deserialización JSON predeterminada en Spring Boot.

Aquí te muestro cómo puedes modificar tus entidades para usar estas anotaciones:

```
import com.fasterxml.jackson.annotation.JsonManagedReference;

@Entity
@Data
```

```

@Table(name = "pedido")
public class Pedido {

    // ...otros campos...

    @OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL)
    @JsonManagedReference
    private List<LineaPedido> lineasPedido;

    // ...otros métodos...
}

import com.fasterxml.jackson.annotation.JsonBackReference;

@Entity
@Data
@Table(name = "linea_pedido")
public class LineaPedido {

    // ...otros campos...

    @ManyToOne
    @JoinColumn(name = "pedido_id")
    @JsonBackReference
    private Pedido pedido;

    // ...otros métodos...
}

```

En este código, `@JsonManagedReference` se coloca en el lado de la relación que quieres serializar, y `@JsonBackReference` se coloca en el lado de la relación que quieres omitir durante la serialización.

Esto debería resolver el problema de recursión infinita al serializar estas entidades a JSON.

El código

```

@EqualsAndHashCode.Exclude @ToString.Exclude
@JsonManagedReference
@Builder.Default
@OneToMany(mappedBy = "pedido", cascade = CascadeType.ALL, orphanRemoval = true)
private Set<LineaPedido> lineas = new HashSet<>();

```

también debería funcionar para evitar la recursión infinita al serializar a JSON. La anotación `@JsonManagedReference` en `lineas` indica que este lado de la relación se debe serializar normalmente, y el otro lado de la relación (en la entidad `LineaPedido`) debería tener la anotación `@JsonBackReference`.

Las anotaciones `@EqualsAndHashCode.Exclude` y `@ToString.Exclude` son de Lombok y se utilizan para excluir el campo `lineas` de los métodos `equals()`, `hashCode()` y `toString()` generados automáticamente. Esto puede ser útil para evitar problemas con las relaciones bidireccionales en estos métodos.

La anotación `@Builder.Default` se utiliza con el constructor de Lombok's Builder para indicar que el campo `lineas` debe inicializarse con un nuevo `HashSet` cuando se construye una instancia de `Pedido` utilizando el patrón Builder.

Por último, `orphanRemoval = true` en la anotación `@OneToMany` significa que si una `LineaPedido` es eliminada de la colección `lineas`, también será eliminada de la base de datos. Esto puede ser útil para mantener la coherencia en tu base de datos.

Este código añade algunas funcionalidades adicionales en comparación con el código que te proporcioné anteriormente, pero ambas versiones deberían evitar la recursión infinita al serializar a JSON.

### 1.3.1 SET y LIST

La elección entre `List` y `Set` depende de tus necesidades específicas.

- `List` es una colección ordenada de elementos que permite duplicados. Mantiene el orden de inserción, lo que significa que puedes acceder a los elementos en el orden en que fueron insertados en la lista.
- `Set` es una colección no ordenada de elementos únicos. No permite duplicados y no garantiza ningún orden específico de sus elementos.

En el contexto de JPA/Hibernate:

- Si usas `List`, Hibernate utilizará una tabla de unión para manejar la relación `OneToMany` y permitirá duplicados. También podrás obtener las entidades en el orden en que fueron insertadas.
- Si usas `Set`, Hibernate también utilizará una tabla de unión, pero no permitirá duplicados.

Por lo tanto, si no quieres permitir duplicados y no te importa el orden de los elementos, puedes usar **Set**. Si quieres mantener el orden de inserción o permitir duplicados, deberías usar **List**.

La elección entre **List** y **Set** no tiene un impacto directo en la recursividad al serializar a JSON. Ambas son interfaces de la colección Java y pueden ser serializadas a JSON. La recursividad infinita se produce debido a las relaciones bidireccionales entre las entidades, no debido al tipo de colección que estás utilizando.

Las anotaciones **@JsonManagedReference** y **@JsonBackReference** de la biblioteca Jackson, o **@JsonIgnore** en ciertos casos, se utilizan para manejar la serialización de estas relaciones bidireccionales y prevenir la recursividad infinita.

Por lo tanto, puedes elegir usar **List** o **Set** basándote en tus necesidades específicas en términos de comportamiento de la colección (como el orden de los elementos y la permisividad de los duplicados), y no necesitas preocuparte por el impacto en la recursividad de la serialización a JSON.